Sistemes Operatius

Tema 3: Sincronització i semàfors · Curs 2026-27

LA SALLE
UNIVERSITAT RAMON LLULL
ESCOLA TÈCNICA SUPERIOR D'ENGINYERIA

Semàfors i mutex: eina segons el context

Processos o fork

Exclusió mútua: semàfor inicialitzat a 1.

Sincronització: semàfor inicialitzat a 0 o amb N permisos.

Threads

Exclusió mútua: pthread_mutex.

Sincronització: semàfors.

Semàfor: definició

```
tipus
  semàfor = registre
    compt : enter
    cua   : cua de processos bloquejats
  firegistre
fitipus

var
  s : semàfor
fivar
```

- compt indica els permisos disponibles.
- cua conté els processos bloquejats.
- El programa no modifica aquests camps directament.
- init, wait i signal han de ser atòmiques.

Semàfor: init, wait i signal

```
proc init(ref S: semàfor, valor: enter)
  S.compt := valor
  S.cua := buida
fiproc
proc wait(ref S: semàfor)
  si S.compt > 0 llavors
    S.compt := S.compt - 1
  altrament
    encuar(S.cua, procés_actual)
    bloquejar(procés_actual)
  fisi
fiproc
```

```
proc signal(ref S: semàfor)
  si no buida(S.cua) llavors
    p := desencuar(S.cua)
    desbloquejar(p)
  altrament
    S.compt := S.compt + 1
  fisi
fiproc
```

wait consumeix un permís o bloqueja; signal desperta un procés o deixa un permís disponible.

Ús 1: exclusió mútua

```
var
  x : enter
  s : semàfor
fivar
x := 7
init(s, 1)
procés A()
  ...
  wait(s)
  x := x + 3
  signal(s)
  ...
fiprocés
```

```
procés B()
  ...
  wait(s)
  x := x - 5
  signal(s)
  ...
fiprocés
```

init(s, 1): només un procés pot executar alhora el fragment que modifica x.

Ús 2: sincronització

```
var
  x : enter
  s : semàfor
fivar

init(s, 0)

procés A()
  ...
  llegeix(x)
  signal(s)
  ...
fiprocés
```

```
procés B()
  var r : real
  fivar

  ...
  wait(s)
  r := sqrt(x)
  escriu(r)
  ...
fiprocés
```

init(s, 0): B no pot utilitzar x fins que A l'ha produït i ha executat signal(s).

Semàfors en C: API del curs

```c
#include "semaphore_v2.h"

typedef struct {
  int shmid;
} semaphore;

int SEM_constructor(semaphore *sem);
int SEM_constructor_with_name(semaphore *sem, key_t key);
int SEM_init(const semaphore *sem, int valor);
int SEM_wait(const semaphore *sem);
int SEM_signal(const semaphore *sem);
int SEM_destructor(const semaphore *sem);
```

- constructor crea o obté l'objecte.
- init fixa el valor inicial una sola vegada.
- wait i signal s'utilitzen durant l'execució.
- destructor elimina el semàfor quan ja no queden usuaris.
- Cal comprovar tots els valors de retorn.

Semàfors en C: ús mínim

```c
#include "semaphore_v2.h"

semaphore sem;

int main(void) {
  SEM_constructor(&sem);
  SEM_init(&sem, 1);

  SEM_wait(&sem);
  /* regió crítica */
  SEM_signal(&sem);

  SEM_destructor(&sem);
  return 0;
}
```

1. Construir.
2. Inicialitzar.
3. Fer wait abans del fragment protegit.
4. Fer signal en sortir.
5. Destruir només al final.

Restaurant: semàfors i valors inicials

```
var
  sem_amanida   : semàfor
  sem_cambrer   : semàfor
fivar

init(sem_amanida, 1)
init(sem_cambrer, 0)
```

sem_amanida = 1

Exclusió mútua sobre el bol d'amanida.

sem_cambrer = 0

Compta els avisos dels clients que han acabat.

Restaurant: pseudocodi del client

```
procés Client()
  per i := 1 fins 10 fer
    menja_sopa()

    wait(sem_amanida)
    menja_amanida()
    signal(sem_amanida)
  fiper

  signal(sem_cambrer)
fiprocés
```

- wait/signal protegeixen només l'amanida.
- Menjar sopa no necessita exclusió mútua.
- En acabar, cada client deixa un avís persistent al semàfor del cambrer.

Restaurant: pseudocodi del cambrer

```
procés Cambrer()
  wait(sem_cambrer)
  wait(sem_cambrer)
  recollir_plats()
fiprocés
```

Hi ha dos clients.

Cada signal(sem_cambrer) representa un client que ha acabat.

El cambrer espera els dos avisos abans de recollir.

Restaurant en C: cambrer, capçaleres i missatges

```c
#include "semaphore_v2.h"
#include <string.h>
#include <unistd.h>

#define MSG_WAIT "Cambrer: esperant els clients...\n"
#define MSG_CLIENT "Rebut l'avís d'un client.\n"
#define MSG_END "Recollint els plats.\n"
```

El cambrer i els clients inclouen la mateixa llibreria i comparteixen les mateixes claus de semàfor.

Restaurant en C: cambrer, crear i inicialitzar

```c
int main(void) {
  semaphore sem_amanida;
  semaphore sem_cambrer;

  key_t ka = ftok("Client.c", 'a');
  key_t kc = ftok("Client.c", 'b');

  SEM_constructor_with_name(&sem_amanida, ka);
  SEM_constructor_with_name(&sem_cambrer, kc);

  SEM_init(&sem_amanida, 1);
  SEM_init(&sem_cambrer, 0);
```

- Client.c ha d'existir perquè ftok funcioni.
- El procés coordinador crea i inicialitza.
- Els clients obtenen els mateixos semàfors, però no els reinicialitzen.

Restaurant en C: cambrer, esperar i destruir

```
    write(1, MSG_WAIT, strlen(MSG_WAIT));

    SEM_wait(&sem_cambrer);
    write(1, MSG_CLIENT, strlen(MSG_CLIENT));

    SEM_wait(&sem_cambrer);
    write(1, MSG_CLIENT, strlen(MSG_CLIENT));

    write(1, MSG_END, strlen(MSG_END));

    SEM_destructor(&sem_amanida);
    SEM_destructor(&sem_cambrer);
    return 0;
}
```

Dos wait perquè hi ha dos clients.

El coordinador elimina els semàfors només després de rebre tots dos avisos.

Restaurant en C: client, capçaleres i constants

```c
#include "semaphore_v2.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define SOPA 5
#define T_MENJA_AMANIDA 4
#define MSG_AMANIDA "Menjant amanida...\n"
#define MSG_END "Sortint del restaurant.\n"
```

Cada client s'executa com un procés independent i rep la seva velocitat com a argument.

Restaurant en C: client, obtenir els semàfors

```
int main(int argc, char *argv[]) {
  int velocitat = atoi(argv[1]);
  semaphore sem_amanida;
  semaphore sem_cambrer;

  key_t ka = ftok("Client.c", 'a');
  key_t kc = ftok("Client.c", 'b');

  SEM_constructor_with_name(&sem_amanida, ka);
  SEM_constructor_with_name(&sem_cambrer, kc);
```

Els dos clients calculen les mateixes claus i obtenen els objectes creats pel coordinador.

No fan SEM_init ni SEM_destructor.

Restaurant en C: client, secció crítica i avís

```
  for (int i = 0; i < SOPA; i++) {
    printf("Menjant sopa %d\n", i);
    sleep(velocitat);

    SEM_wait(&sem_amanida);
    write(1, MSG_AMANIDA, strlen(MSG_AMANIDA));
    sleep(T_MENJA_AMANIDA);
    SEM_signal(&sem_amanida);
  }

  SEM_signal(&sem_cambrer);
  write(1, MSG_END, strlen(MSG_END));
  return 0;
}
```

La secció crítica és tan curta com sigui possible.

L'últim signal no protegeix cap recurs: notifica al cambrer que aquest client ha acabat.

Resum: quina eina fem servir?

Processos o fork

Exclusió mútua: semàfors.

Sincronització: semàfors.

Threads

Exclusió mútua: pthread_mutex.

Sincronització: semàfors.